

Name: _____
Surname: _____
Date: _____

Instructions

Always assume each code is run independently, so variables do not persist between exercises.

Questions 1-6: Multiple Choice (5 points each)

Circle the correct answer for each question, as each question has **only one** correct answer. A correct answer awards 5 points. There is no penalty for wrong choices.

Questions 7-9: Debugging (10 points each)

The functions below contain one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions 10-11: Open Problems (20 points each)

In the following exercises, you will encounter problems that are more open-ended and may not have a single correct answer. Your task is to explain how you would approach each problem by breaking it down into smaller, manageable steps.

Describe how you would structure your code, which logical steps are needed, and what kind of Python tools or constructs (e.g., loops, conditionals, functions, lists, etc.) you would use. You are not required to write full code, but doing so is encouraged if it helps clarify your ideas.

1. Which of these statements is **TRUE** about **integer indexing** with arrays?
 - (a) Integer indexing allows selecting arbitrary elements of an array using a list or array of integer positions.
 - (b) Integer indexing can only be used when the array is sorted.
 - (c) Integer indexing can only be applied to one-dimensional arrays.
 - (d) Integer indexing modifies the original array in place.

2. After running the following code, what will be the value of “str2”?

```
1 str1 = "Merry Christmas"
2
3 str2 = ""
4 vowels = "aeiou"
5 for char in str1:
6     if char in vowels:
7         continue
8     elif char == " ":
9         str2 += "-"
10    else:
11        str2 += char.lower()
```

- (a) str2 = ‘‘M’’
- (b) str2 = ‘‘mrry-chrstms’’
- (c) str2 = ‘‘merry-christmas’’
- (d) The code will raise an error

3. Which of the following statements about mutability in Python is **FALSE**?

- (a) Modifying a list inside a function also modifies it outside the function, even without returning it.
- (b) If we define a default parameter for a function as an empty list [], a new empty list is created on every function call.
- (c) We can modify a list that contains tuples, but we cannot modify the tuples inside the list.
- (d) To create an independent copy of a list, assigning it to another variable is not enough; we must use `.copy()` or another copying method.

4. Which of the following lines will raise an error?

```
1 coordinates = ((0, 0), (0,1), (1, 0), (1, 1))
2 new_coord = []
3 new_coord.append(coordinates[2])    #Line 3
4 print(f"Last coordinate {coordinates[-1]}")    #Line 4
5 coordinates[0] = (coordinates[0][0] - 1, coordinates[0][1] - 1) #Line 5
```

- (a) Line 3.
- (b) Line 4.
- (c) Line 5.
- (d) None of them.

5. After running this code, what will be the final value of “common_holidays”?

```
1 holidays1 = [1, 4, 12, 13, 14, 17, 23, 24]
2 holidays2 = [4, 5, 6, 7, 24, 25, 26, 31]
3 holidays3 = [1, 2, 3, 24, 26, 27, 28]
4
5 common_holidays = set(holidays1) & set(holidays2) - set(holidays3)
6
7 common_holidays = [str(day) + " of December" for day in common_holidays]
8
9 print(common_holidays)
```

- (a) common_holidays = ["24 of December", "4 of December"]
- (b) common_holidays = ["4 of December"]
- (c) common_holidays = [4, "of December"]
- (d) The code will raise an error.

6. What will be the value of array B after applying the function manipulate_arr()?

```
1 import numpy as np
2
3 def manipulate_arr(arr: np.ndarray)->np.ndarray:
4
5     C = np.arange(3) + 10
6
7     D = np.hstack((arr, C.reshape(C.size, 1)))
8
9     E = np.vstack((D, C))
10
11     return E
12
13 A = np.array([[1, 2, 3],
14               [4, 5, 6],
15               [7, 8, 9]])
16
17 B = manipulate_arr(A)
```

(a)
$$\begin{pmatrix} 1 & 2 & 3 & 10 \\ 4 & 5 & 6 & 10 \\ 7 & 8 & 9 & 10 \\ 10 & 10 & 10 & 10 \end{pmatrix}$$

(b)
$$\begin{pmatrix} 1 & 2 & 3 & 10 \\ 4 & 5 & 6 & 11 \\ 7 & 8 & 9 & 12 \end{pmatrix}$$

(c)
$$\begin{pmatrix} 1 & 2 & 3 & 10 \\ 4 & 5 & 6 & 11 \\ 7 & 8 & 9 & 12 \\ 10 & 11 & 12 & 0 \end{pmatrix}$$

- (d) The code will raise an error because shapes do not match.

7. The following code is intended to decrypt messages that were encrypted as follows: wherever the original message contains a space, a dollar sign \$ is placed instead, and extra spaces are inserted between words where there were none. To decrypt it, the \$ symbols should be replaced by spaces and the extra spaces removed. However, the code does not produce the desired output. For example, it returns a list instead of a string. Identify the four errors and correct them.

```
1 def decrypt_msg(message: str)->str:
2     #Split message without "$"
3     message_list = message.split(" ")
4
5     #Iterate over message and replace $
6     decrypt_msg = []
7     for word in message_list:
8         word.replace("$", "")
9         decrypt_msg.append(word)
10
11     return decrypt_msg
12
13 input_message = "I$a m$ d oi ng$ a$ g r ea t$ex am"
14
15 print(decrypt_msg(input_message))
16
17 #Expected output
18 #"I am doing a great exam"
19
20 #Actual output
21 #['I$a', 'm$', 'd', 'oi', 'ng$', 'a$', 'g', 'r', 'ea', 't$ex', 'am']
```

8. This code is intended to simulate the random walks of two particles in two dimensions with the same number of steps and compare their final positions. The function raises an error on line 14: “ValueError: The truth value of an array with more than one element is ambiguous”. Identify the two errors and correct them.

```
1 import numpy as np
2
3 def random_walk_2D(directions: list[tuple], num_steps: int)->bool:
4     #Generate two random arrays of directions
5     pos_arr1 = rng.choice(directions, size = num_steps)
6     pos_arr2 = rng.choice(directions)
7
8     #Calculate the final position [posx, posy] of each particle
9     #computing the cumsum of each column
10    final_pos1 = np.cumsum(pos_arr1, axis = 0)
11    final_pos2 = np.cumsum(pos_arr2, axis = 0)
12
13    #Compare the final position
14    if final_pos1[-1] == final_pos2[-1]:
15        return True
16
17    else:
18        return False
19
20 #Create list of directions
21 directions = [(0, 0), (0, 1), (1, 0), (0, -1), (-1, 0)]
22
23 rng = np.random.default_rng(40)
24
25 random_walk_2D(directions, 10)
```

9. This code is intended to create a 2-D array A from an array B with 2 columns and an odd number of rows, placing the two columns of B into A in the shape of a cross (see the expected output). The function raises an error on line 15: “ValueError: too many indices for array: array is 1-dimensional, but 2 were indexed”. Identify the three errors and correct them.

```
1 import numpy as np
2
3 def cross_matrix(B: np.ndarray)->np.ndarray:
4     #Get number of rows
5     n = B.shape[1]
6
7     #Create an array filled with zeros
8     A = np.zeros(n)
9
10    #Extract both columns
11    column1 = B[0, :]
12    column2 = B[:, 1]
13
14    #Add the columns as a cross to the new array
15    A[:, n//2] = column1
16    A[n//2, :] = column2
17
18    return A
19
20 B = np.array([[1, 4],
21               [2, 5],
22               [3, 6]])
23
24 print(f"Cross array:\n {cross_matrix(B)}")
25
26 #Expected output
27 #Cross array:
28 #[[0. 1. 0.]
29  #[4. 5. 6.]
30  #[0. 3. 0.]
```

10. In engineering applications, it is common to work with sensors that report precision along three axes (X, Y, and Z). Over time, sensors lose precision on all axes, and only those whose precision remains above a required threshold should be kept.

In this setting, the data is represented as a 2D array (matrix), where each row corresponds to a sensor and each column corresponds to its precision on one axis. Each year, every sensor loses a random amount of precision between 0 and 0.2, and this loss is applied to all three axes of that sensor. The yearly precision update for each value can be expressed as:

$$\text{sensor_precision}' = \text{sensor_precision} \cdot (1 - \text{precision_loss})$$

where `precision_loss` contains the randomly generated precision loss for each sensor.

After applying this yearly loss, sensors whose precision on all three axes remains strictly above 70% are kept, whereas any sensor that falls below this threshold has its entire row replaced with zeros. This degradation-and-filtering process is repeated each year.

Create a function called `filter_sensors_by_precision()`. It takes three inputs:

- **sensor_precision**: a 2D NumPy array where each row represents a sensor and each column represents its precision on one axis.
- **minimum_precision**: the threshold below which a sensor is discarded.
- **years**: the number of consecutive years over which precision degradation is applied.

The function must return a 2D NumPy array in which sensors that remain above the threshold on all three axes keep their updated precision values, while sensors that fall below the threshold (**in any axis**) have their entire row set to 0.

If all sensors fall below the threshold, the function should print a message indicating that no sensors maintain the required precision after the given period.

```
1 sensor_precision = np.array([
2     [96.85, 95.25, 85.47],
3     [97.43, 92.12, 95.48],
4     [86.32, 90.05, 75.41],
5     [90.12, 85.15, 80.00],
6 ]) # each row is a sensor, columns are precisions on X, Y, Z
7
8 possible_expected_output = np.array([
9     [0, 0, 0],
10    [0, 0, 0],
11    [65.30516806, 68.12708971, 57.05123637],
12    [0, 0, 0]]
13 )
```


11. You are a programmer who wants to create a task-management tool. The starting point is a weekly task schedule stored as a nested list. Each element contains a day followed by tuples of the form (hour, task). (See the example below)

To begin, you will create a function called `tasks_list_to_dict()` that allows direct access to the tasks of each day. The function receives the nested list `tasks_week` as input and converts this structure into a dictionary of dictionaries, where the keys are day names and the values are dictionaries mapping hours to tasks.

After that, you want to be able to add new tasks while checking whether a task already exists at the given time. To do this, ask the user for the day, hour, and task they want to add.

Then create another function, `add_task()`, which checks whether there is already a task in that time slot: if there is, it prints a message saying that a task already exists at that hour; if not, it adds the new task to the dictionary.

The function `add_task` takes the following inputs:

- `task_dict`: The dictionary produced by `tasks_list_to_dict()`
- `day`: The day entered by the user as a string
- `hour`: The hour entered by the user as a string
- `task`: The task entered by the user as a string

```
1 example_input = [  
2     ["Monday", ("09:00", "Check emails"), ("11:00", "Team meeting")],  
3     ["Tuesday", ("08:30", "Exercise"), ("10:00", "Write report")],  
4     ["Wednesday", ("09:00", "Team session"), ("12:00", "Work lunch")],  
5     ["Thursday", ("09:30", "Documentation"), ("13:00", "Meeting")],  
6     ["Friday", ("08:00", "New planning"), ("11:30", "Review report")]  
7 ]  
8  
9  
10 #Output 1: Dictionary of dictionaries containing the tasks scheduled for  
    each day and hour.  
11 #Output 2: Dictionary updated with the newly added tasks (or a warning  
    message if a task already exists at that hour).
```


